

Festival

Nayra pelaa festivaaleilla peliä, jonka pääpalkintona on matka Laguna Coloradalle. Pelissä käytetään merkkejä kuponkien ostamiseen. Kupongin ostaminen saattaa johtaa lisämerkkeihin. Tavoitteena on kerätä mahdollisimman monta kuponkia.

Pelin alussa Nayralla on A merkkiä. Pelissä on N kuponkia, numeroituna luvuilla 0:sta lukuun $N - 1$. Ostaakseen kupongin i ($0 \leq i < N$) Nayran täytyy maksaa $P[i]$ merkkiä (ja hänellä täytyy olla vähintään $P[i]$ merkkiä ennen kauppaa). Hän voi ostaa kunkin kupongin korkeintaan kerran.

Lisäksi jokaisella kupongilla i ($0 \leq i < N$) on **tyyppi** $T[i]$, joka on kokonaisluku **suljetulta välillä 1:stä 4:ään**. Kun Nayra on ostanut kupongin i , hänen jäljellä olevien kuponkiensa määrä kerrotaan luvulla $T[i]$. Muodollisesti, jos hänellä on X merkkiä jossain pelin vaiheessa ja hän ostaa kupongin i (mikä vaatii, että $X \geq P[i]$), niin hänellä on $(X - P[i]) \cdot T[i]$ merkkiä ostoksen jälkeen.

Tehtäväsi on määrittää, mitkä kupongit Nayran tulisi ostaa ja missä järjestyksessä, jotta hän maksimoisi hänellä lopussa olevien **kuponkien** kokonaismäärän. Jos on enemmän kuin yksi jono ostoksia, joilla tämän voi saavuttaa, voit ilmoittaa minkä tahansa niistä.

Implementaatioyksityiskohdat

Sinun tulee toteuttaa seuraava aliohjelma.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : Nayralla aluessa olevien merkkien määrä.
- P : N :n alkion taulukko kuponkien hinnoista.
- T : N :n alkion taulukko kuponkien tyypeistä.
- Tätä aliohjelmaa kutsutaan tasan kerran kutakin testiä kohden.

Aliohjelman tulee palauttaa taulukko R , joka kertoo Nayran ostokset seuraavasti:

- Taulukon R pituuden tulee vastata suurinta mahdollista määrää kuponkeja, jonka Nayra voi ostaa.
- Taulukon alkiot ovat niiden kuponkien indeksit, jotka hänen tulee ostaa, kronologisessa järjestyksessä. Eli hän ostaa ensimmäisenä kupongin $R[0]$, toisena kupongin $R[1]$ ja niin edelleen.
- Taulukon R alkioden tulee olla toisistaan eroavia.

Taulukon R tulee olla tyhjä, jos ei ole mahdollista ostaa yhtäkään kuponkia.

Rajat

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ jokaiselle i , jolle $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ jokaiselle i , jolle $0 \leq i < N$.

Osatehtävät

Osatehtävä	Pisteet	Lisärajoitukset
1	5	$T[i] = 1$ jokaiselle i , jolle $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ jokaiselle i , jolle $0 \leq i < N$.
3	12	$T[i] \leq 2$ jokaiselle i , jolle $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra voi ostaa kaikki N kuponkia (jossain järjestyksessä).
6	16	$(A - P[i]) \cdot T[i] < A$ jokaiselle i , jolle $0 \leq i < N$.
7	18	Ei lisärajoitukset.

Esimerkit

Esimerkki 1

Tarkastellaan seuraavaa kutsua.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayralla on aluksi $A = 13$ merkkiä. Hän voi ostaa 3 kuponkia alla olevassa järjestyksessä:

Ostettu kuponki	Kupongin hinta	Kupongin tyyppi	Merkkien määrä ostoksen jälkeen
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Tässä esimerkissä Nayran ei ole mahdollista ostaa enempää kuin 3 kuponkia, ja yllä kuvattu ostosten jono on ainoa tapa, jolla hän voi ostaa niitä 3. Täten aliohjelman tulee palauttaa $[2, 3, 0]$.

Esimerkki 2

Tarkastellaan seuraavaa kutsua.

```
max_coupons(9, [6, 5], [2, 3])
```

Tässä esimerkissä Nayra voi ostaa molemmat kupongit missä järjestyksessä tahansa. Täten aliohjelman tulee palauttaa joko $[0, 1]$ tai $[1, 0]$.

Esimerkki 3

Tarkastellaan seuraavaa kutsua.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Tässä esimerkissä Nayralla on yksi merkki, joka ei riitä yhdenkään kupongin ostamiseen. Täten aliohjelman tulee palauttaa $[]$ (tyhjä taulukko).

Malliarvostelija

Syötteen muoto:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Tulosteen muoto:

```
S
R[0] R[1] ... R[S-1]
```

Tässä S on aliohjelman `max_coupons` palauttaman taulukon R pituus.